BUILD IT.
UNDERSTAND IT.
EXPLAIN IT.

Your project engineering handbook

Folio / Sentinel / IP Location Service

A practical guide to the code, concepts, tradeoffs, tests, and interview conversations behind your projects.

Prepared for Amrit Raj | October 2026

How to use this guide

Read a chapter, trace its code, predict an outcome, run the exercise, and explain the result without notes. The code is the source of truth. Known limitations are included so you can discuss your work accurately.

Learning map

1. What you are building

2. Foundations: follow one HTTP request

3. Sentinel: architecture and code map

4. Sentinel: understand the algorithm

5. Sentinel: security, failures, and metrics

6. Folio: the RAG pipeline

7. Folio: code, storage, and ownership

8. Folio: ingestion and reliability

9. Folio: evidence and evaluation

10. Location service: the accurate story

11. Tests: prove behavior

12. Run and deploy

13. Frontend engineering

14. Interview walkthroughs

15. Learning and contribution plan

16. Resume and demo checklist

The editable Markdown companion contains the same material. Revisit chapters 4, 7, 9, and 10 before interviews: these contain the most important reasoning and tradeoffs.

This guide explains the actual projects in this workspace. Read a chapter, trace the named files, run the exercise, and explain the result aloud. Features described as future work are not implemented. The new versions are Folio (RAG assistant) and Sentinel (API rate limiter). The IP location service retains its existing scope.

1. What you are building

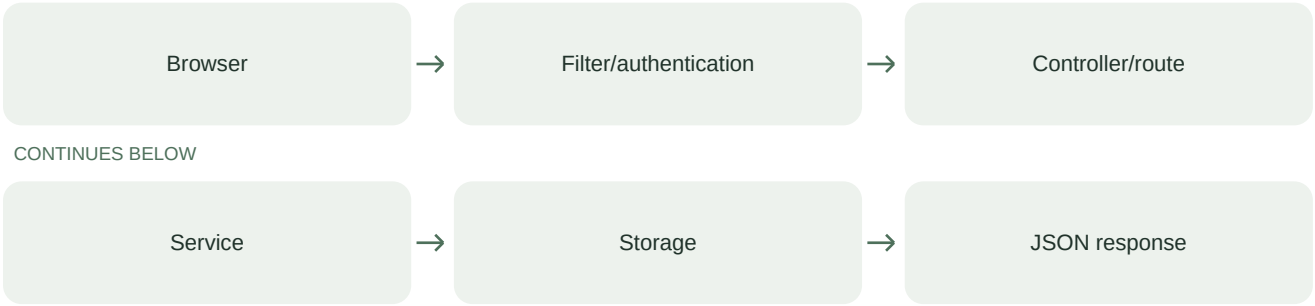
Project	User problem	Engineering idea	Honest boundary
Folio	Finding evidence inside PDFs is slow	Retrieve relevant passages, then answer with source references	Retrieval can miss evidence; citations are not proof of truth
Sentinel	One client can overwhelm an API	Atomically enforce burst and sustained quotas in Redis	A demonstration service, not a full reverse proxy or DDoS shield
IP location	Convert a public IP into geographic metadata	Local MaxMind lookup with cache and isolated audit logging	Database approximation, not GPS or exact personal location

Your strongest interview story is the chain from problem to design, failure case, test, and tradeoff. Do not memorise technology names without understanding why each dependency exists. A smaller system you understand deeply is more credible than an inflated production-scale claim.

The two upgrades stay within a fresher's reach: plain HTML/CSS/JavaScript frontends, existing Java/Python backends, and carefully tested architectural additions. You do not need Kubernetes or Kafka to demonstrate concurrency, security, and reliability.

2. Foundations: follow one HTTP request

A browser is a client. It sends an HTTP request containing a method, path, headers, and sometimes a body. GET reads data, POST submits work, PUT replaces configuration, and DELETE removes a resource. JSON is a text format for structured data. HTTP 200 means success; 202 means accepted, not completed; 400/422 indicate invalid input; 401 means authentication failed; 404 means missing or inaccessible; 429 means too many requests; 503 means a required service is unavailable.



The controller translates HTTP into a service call. The service contains business rules. The repository reads and writes persistent records. A DTO or Pydantic model defines input/output shape. Separating these responsibilities makes rules testable without starting the whole application.

Dependency injection means a framework supplies an object's collaborators instead of the object creating them itself. Spring creates services and repositories as beans and injects them through constructors. FastAPI Depends resolves dependencies such as the current user or database session. Tests can replace a collaborator with a controlled fake.

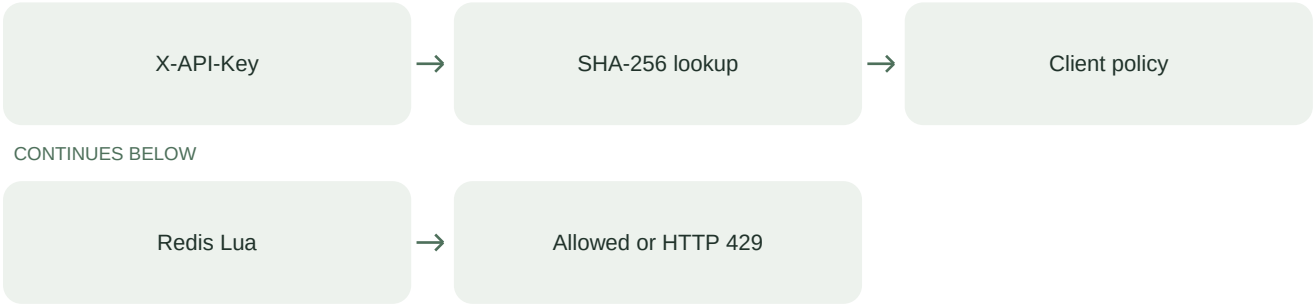
Persistence means data survives a process restart. MySQL and SQLite are databases; Redis stores temporary counters and cache entries here; FAISS indexes vectors for search. A Java/Python variable disappears when its process exits. A file survives only if the deployment filesystem or mounted disk survives.

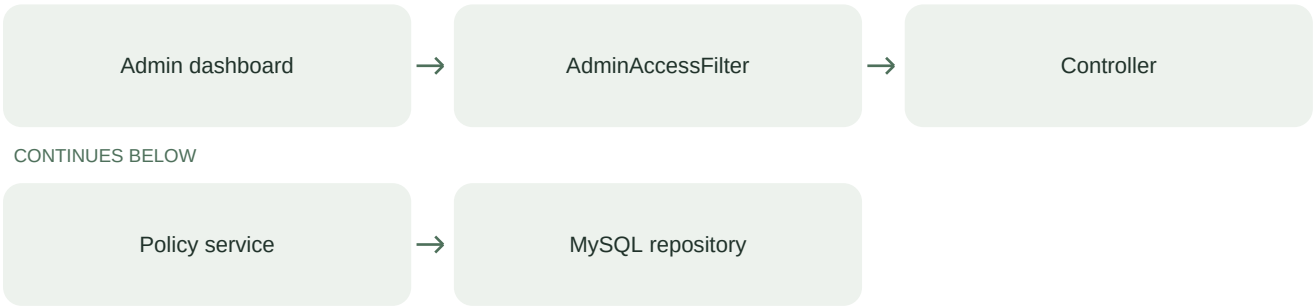
Concept	Concrete example	Why it matters
Authentication	Verify a JWT or hash an API key	Establish who is calling
Authorization	Restrict document IDs to the owner	Decide what the caller may access
Validation	Limit a question to 2,000 characters	Reject unsuitable input before expensive work
Atomicity	Check both quotas and add permits in Lua	Prevent partial decisions and races
Transaction	Commit a client policy to SQL	Keep database changes consistent
Timeout	Stop waiting for an embedding provider	Bound resource occupation
Observability	Record outcomes and latency	Understand what the system did

Exercise: open the browser Network panel, send one request, and identify the method, path, status, body, and response. Explain a transport failure versus a valid HTTP 429 response. A 429 proves the server responded; it is not a network outage.

3. Sentinel: architecture and code map

Sentinel extends the Spring Boot limiter with registered clients, hashed API keys, two quota windows, protected administration, and a traffic playground. Anonymous requests use the existing IP limiter. Registered requests supply X-API-Key. Administration uses a separate X-Admin-Key secret configured with at least 32 characters.





File or class	Responsibility	Question it answers
RateLimitFilter	Choose policy, enforce it, add headers, record metrics	Where is a request intercepted?
ClientPolicyService	Validate policies, generate/hash/rotate keys	Why is the key shown only once?
ApiClient / ApiClientRepository	Persist client configuration and hash	What survives a restart?
PolicyLimiter	Execute the two-window Lua decision	Why cannot concurrency overrun quota?
AdminAccessFilter	Guard administration and operational endpoints	Who can change limits?
RateLimiterService	Anonymous limiter and blocklist	What happens without an API key?
TrafficLoggingService	Store request observations	What adds SQL work to the HTTP path?
assets/dashboard.js	Send requests and show session results	Are numbers global or browser-local?

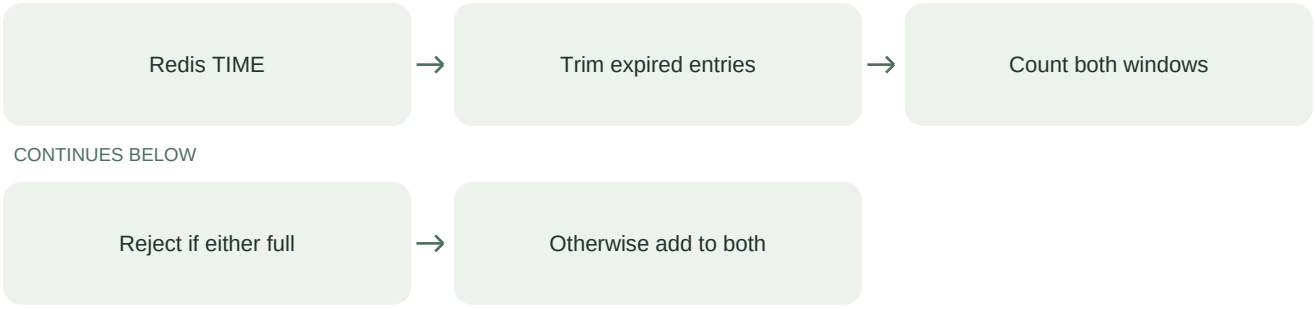
A servlet filter runs before a controller. A rejected request never reaches the protected work. Static assets and the dashboard are exempt from quota enforcement so users can inspect a throttled API. Administrative endpoints are exempt from quotas but protected by the independent admin filter.

The policy table contains ID, name, SHA-256 key hash, burst limit/window, sustained limit/window, enabled flag, and optimistic-lock version. JPA's version field detects conflicting updates instead of silently overwriting a newer row. A SQL authentication error produces 503; it does not silently become an anonymous call.

4. Sentinel: understand the algorithm

A fixed window counts requests in buckets such as 12:00:00-12:00:59. A client could send its quota just before a boundary and again just after it. A sliding log retains timestamps from the immediately preceding duration. Redis sorted sets associate each unique request ID with its timestamp score.

The Lua script gets Redis server time, removes expired timestamps, and counts both windows. If either is full, it rejects without adding a permit to either set. If both have capacity, it adds the same unique request ID to both sets and assigns expiration times. Redis executes the script atomically, so another quota decision cannot interleave its operations.



Time	Attempt	Burst: 3 / 10 seconds	Sustained: 4 / 60 seconds	Result
0 seconds	Request 1	Count becomes 1	Count becomes 1	Allowed
1 second	Request 2	Count becomes 2	Count becomes 2	Allowed
2 seconds	Request 3	Count becomes 3	Count becomes 3	Allowed
3 seconds	Request 4	Already full	Remains 3	Burst rejection
12 seconds	Request 5	Old burst permits expire	Count becomes 4	Allowed
13 seconds	Request 6	Burst has room	Already full	Sustained rejection

The burst window limits short spikes; the sustained window limits continued usage. A client must satisfy both. Remaining quota describes the binding window, not a promise of future capacity. Other requests can consume permits immediately after the response.

UUID members prevent requests in the same millisecond from replacing each other. Redis TIME avoids application-server clock differences. The {client-id} hash tag places both keys in one Redis Cluster slot, although production cluster deployment has not been validated here.

Retry-After estimates the seconds until enough permits expire. If an administrator reduces a limit below the current count, waiting for only the oldest request is insufficient. The script selects the timestamp at count-minus-limit. Clients should wait at least the returned duration; another caller sharing the key can still compete for capacity.

Window durations are fixed after creation. Increasing a window cannot reconstruct history already trimmed under the shorter duration. Create a new client for different durations. Limits and enabled status remain editable. Redis keys use client IDs, so rotating an API key does not grant fresh quota.

Sorted-set insertion costs logarithmic time in retained entries. Trimming also costs work proportional to removed entries. Sliding logs are precise but use more memory than fixed counters or token buckets. Each admitted request occupies an entry in two sets until expiration. Explain this tradeoff rather than calling one algorithm universally best.

5. Sentinel: security, failures, and metrics

An API key is generated from 32 secure random bytes. Only its SHA-256 hash is stored. The server hashes the supplied key and searches for that hash. Hashing suits high-entropy random keys; human passwords need a slow password-hashing algorithm. A database leak should not reveal usable keys directly. The admin secret remains an environment variable and is compared in constant time.

Disabling a client makes authentication fail. Rotation replaces its hash, invalidating the old key for new requests. A request already authenticated before rotation may still complete. Never put real keys in screenshots, source control, URLs, or logs. The dashboard keeps its admin secret in memory and clears the visible input after unlocking.

Redis failure deliberately fails open for quota enforcement to preserve availability. Requests can exceed quotas during an outage; responses are labelled FAIL_OPEN and metrics count this outcome. SQL failure during client authentication fails closed. A payment, login, or abuse-sensitive endpoint might require fail-closed quota enforcement instead.

The code uses the network peer address, not arbitrary X-Forwarded-For values. Otherwise callers could spoof a fresh IP on every request. Behind a proxy, anonymous users may share the proxy address until a trusted-proxy strategy is implemented. Registered clients use their API-key identity for quotas.

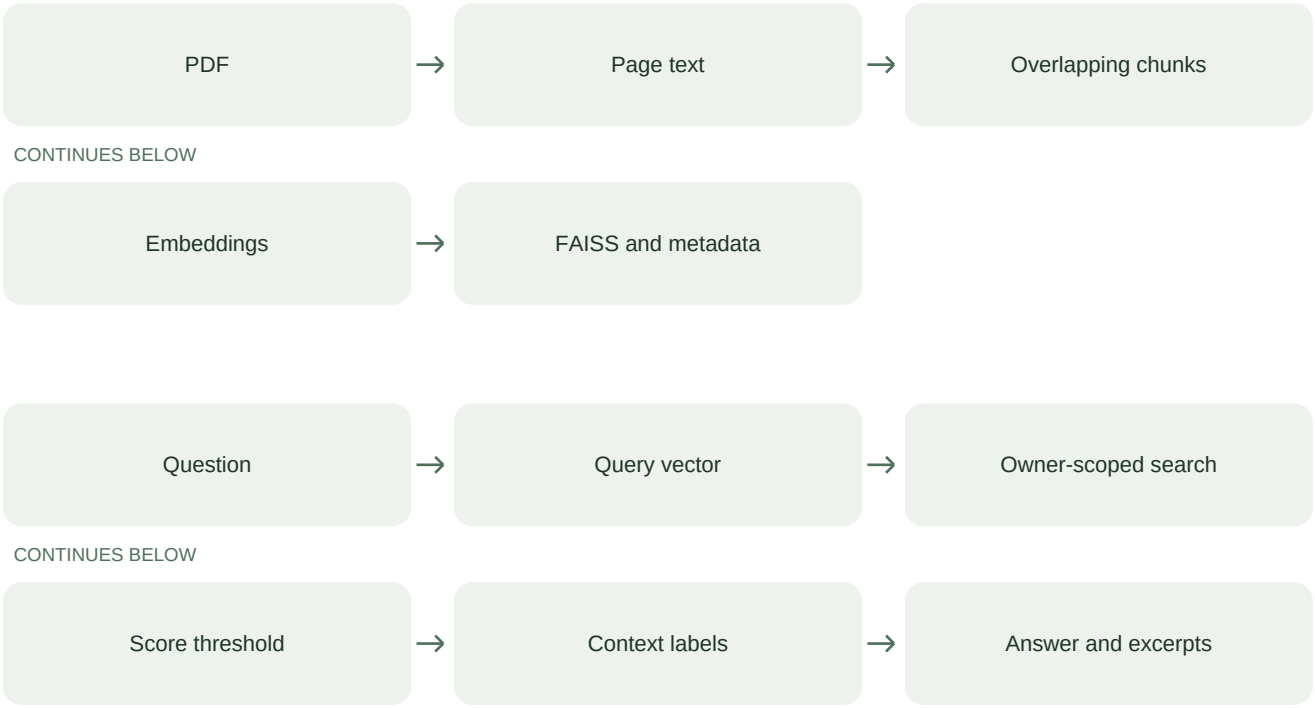
Measurement	Meaning	Common mistake
Throughput	Decisions/responses per second for a stated workload	Counting rejected responses as useful business work
p95 latency	95 percent finish within this duration	Treating p95 as the average
Browser totals	Results sent by this page, capped at 1,000 events	Calling them fleet-wide telemetry
Backend metrics	Outcome counts and decision durations	Using unbounded IP/path labels
Service benchmark	Direct limiter calls	Claiming HTTP, auth, and SQL logging are included

The previous 3,695 allowed decisions/second result measured the earlier anonymous service path: 10 workers, 10,000 calls, Redis 7, approximately 4.46 ms p95. It is not a benchmark of the new authenticated two-window HTTP path. Measure that path separately before claiming its performance.

Exercise: create a 3/10-second burst and 4/60-second sustained policy. Send four quick requests, wait 11 seconds, then send two more. Predict decisions first. Rotate the key and verify authentication changes while quota remains. Explain why refreshing the dashboard clears browser statistics but not Redis counters.

6. Folio: the RAG pipeline

RAG means retrieval-augmented generation. The application retrieves relevant text and supplies it as context to an answer provider. Retrieval and generation are separate stages. You can test retrieval without an LLM, which is how the read-only sample works.



A PDF contains positioned text and graphics. PyMuPDF extracts selectable text page by page. Scanned images need OCR, which this project does not implement. Page numbers let users inspect the original source. Cleanup removes unsuitable spacing and null characters; chunking splits text into manageable passages.

Defaults are 900 characters per chunk with 150 characters of overlap. These are characters, not model tokens. Overlap reduces boundary information loss but increases index size and duplicated context. Smaller chunks isolate facts; larger chunks preserve explanations. Tune using labelled retrieval examples.

An embedding is a numeric representation of text. Hash mode maps words into signed buckets and normalises the vector. It is an inexpensive lexical baseline, not semantic understanding. Synonyms, word forms, and hash collisions can hurt retrieval. Gemini is an optional semantic embedding provider. Document and query vectors must use compatible models, dimensions, and normalisation.

For normalised vectors, the dot product behaves as cosine similarity. It measures alignment, not the probability an answer is correct. FAISS searches vectors; metadata maps each vector to document ID,

filename, page, chunk ID, and text. Switching embedding models requires reindexing even if dimensions match.

7. Folio: code, storage, and ownership

File	Role	Invariant
app/main.py	Routes, OAuth callback, lifecycle	Derive owned IDs from authenticated user
app/auth.py	Sign and verify tokens	Caller cannot choose an arbitrary user ID
app/ingestion.py	Jobs and original-file access	Jobs and files belong to their owner
services/rag_service.py	Orchestrate upload and question answering	No supporting chunks means no LLM call
services/vector_store.py	Persist vectors/metadata and search	Filter ownership before returning context
services/embedding_service.py	Hash or Gemini embeddings	Keep model and dimensions compatible
services/llm_service.py	Mock, Gemini, or OpenAI answers	Treat document text as untrusted data
app/demo.py	Fictional read-only corpus	Never read account documents
static/app.js	Workspace interactions	Render source content as text, not HTML

SQLite stores users, ownership, chat history, and ingestion jobs. FAISS plus metadata/embeddings holds searchable content. Original PDFs live in uploads. Back up these stores together. SQL and filesystem/index changes do not form a distributed transaction; cleanup handles ordinary failures, but a process crash can require reconciliation.

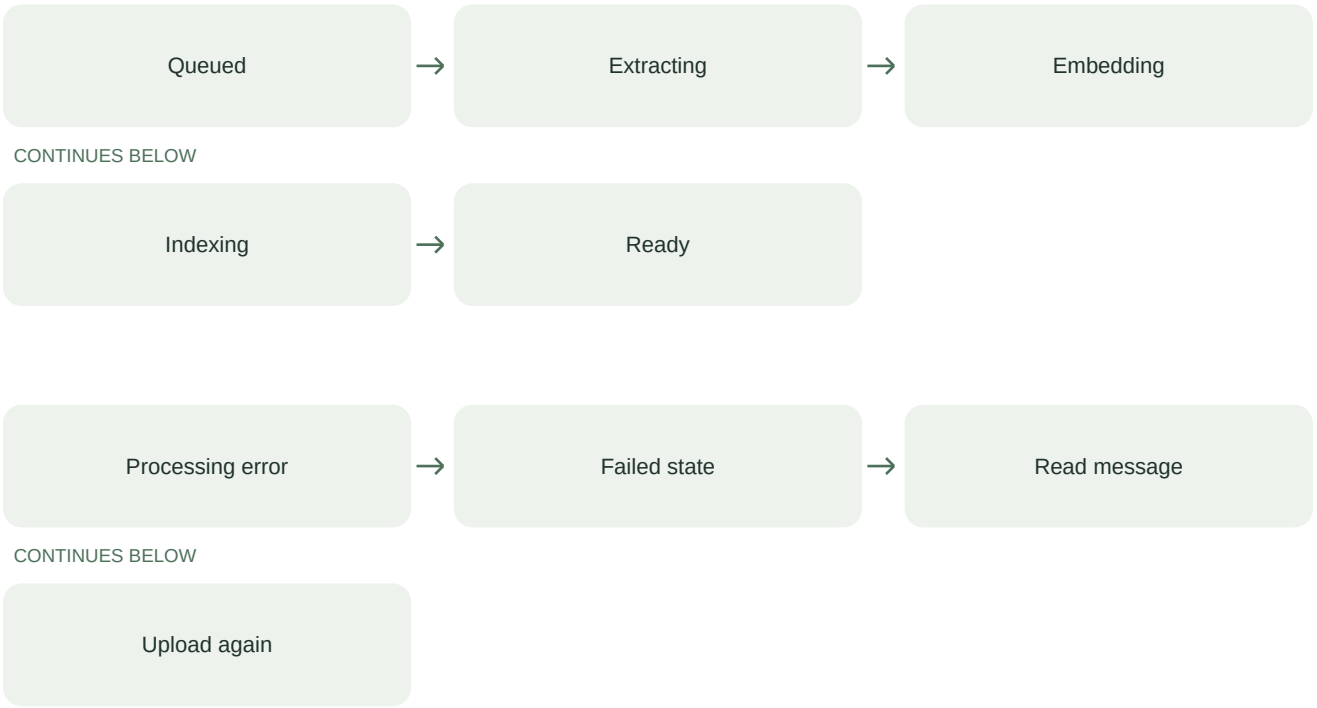
Google OAuth identifies a user through the provider response. Missing or false email verification is rejected. The server creates a signed JWT; the browser keeps it in sessionStorage and sends Authorization: Bearer. Signing detects modification; it does not encrypt the payload. Never place secrets inside JWT claims.

Login alone is not isolation. The server computes owned IDs from SQL. A selected ID outside that set receives 404. Unscoped questions still pass only owned IDs to retrieval. Deletion, original-file access, and job listing also check ownership. A filename never grants authority to read an arbitrary path.

Session storage is accessible to JavaScript, making XSS prevention essential. The frontend uses textContent for filenames, answers, questions, and excerpts. Production needs HTTPS and strong, distinct JWT/OAuth-state secrets. HttpOnly server-side sessions are a possible future hardening step, not implemented here.

8. Folio: ingestion and reliability

The upload endpoint returns HTTP 202 with a job ID. It validates the PDF filename and bounds upload bytes, default 10 MB. The browser polls authenticated status and displays actual stages, not fabricated percentages. A document may spend most of its processing time embedding; two completed stages do not imply 50 percent completion.



State	Work	Failure example
queued	Bytes saved; background task scheduled	Process stops before execution
extracting	Read PDF and create passages	Corrupt or image-only PDF
embedding	Compute numeric representations	Provider timeout or invalid credentials
indexing	Persist searchable vectors	Disk write or dimension error
ready	Ownership committed; document available	Successful terminal state
failed	Processing could not finish	Bounded error shown to user

This is a single-worker, in-process pipeline using FastAPI BackgroundTasks, not an external durable queue. Restart marks unfinished jobs failed. Run one application worker for the file-backed design. Multiple workers require a durable queue, shared storage, and safe index ownership or a suitable vector database.

The one-active-upload check is a single-worker guard, not a distributed lock. Byte limits do not completely bound extracted text or model costs. Page/chunk limits and per-user quotas are useful future hardening. Do not imply these controls already exist.

The recovery test corrupts FAISS while retaining valid saved embeddings and metadata. The index can be rebuilt from those intact inputs; lost source data cannot be reconstructed magically. Backups remain necessary. Recovery is different from replication, and neither alone establishes disaster readiness.

Exercise: upload a short selectable-text PDF, inspect processing, ask a question, and open the original cited page. Try an image-only PDF. Restart during a slow disposable upload and verify its failure state. Use test files, not another person's private documents.

9. Folio: evidence and evaluation

Chunks below MIN_RETRIEVAL_SCORE are discarded. With none remaining, the server returns insufficient evidence without calling the LLM. The default 0.20 is a starting threshold, not universally calibrated. Its usefulness depends on provider and corpus.

Context includes labels such as [S1], filename, and page. The prompt asks for supplied citations and tells the model to ignore instructions inside source documents. This does not guarantee prompt-injection resistance or truth. The frontend exposes excerpts and original pages so users can inspect support. It does not claim citations are semantically verified.

The public sample uses three fictional documents and returns retrieved text directly. It needs neither OAuth nor paid model credentials. Its suggested questions are regression examples, not a statistically meaningful quality benchmark.

Evaluation	Calculation	Diagnoses
Recall at k	Queries with relevant source in top k / answerable queries	Missing useful evidence
Mean reciprocal rank	Average 1 / first relevant rank; 0 if missing	Whether useful sources rank early
Abstention accuracy	Correct abstentions / unanswerable queries	Unsupported questions accepted
Citation validity	Existing cited labels / all cited labels	Invented labels, not factual support
Groundedness review	Human check of claims against excerpts	Unsupported reasoning

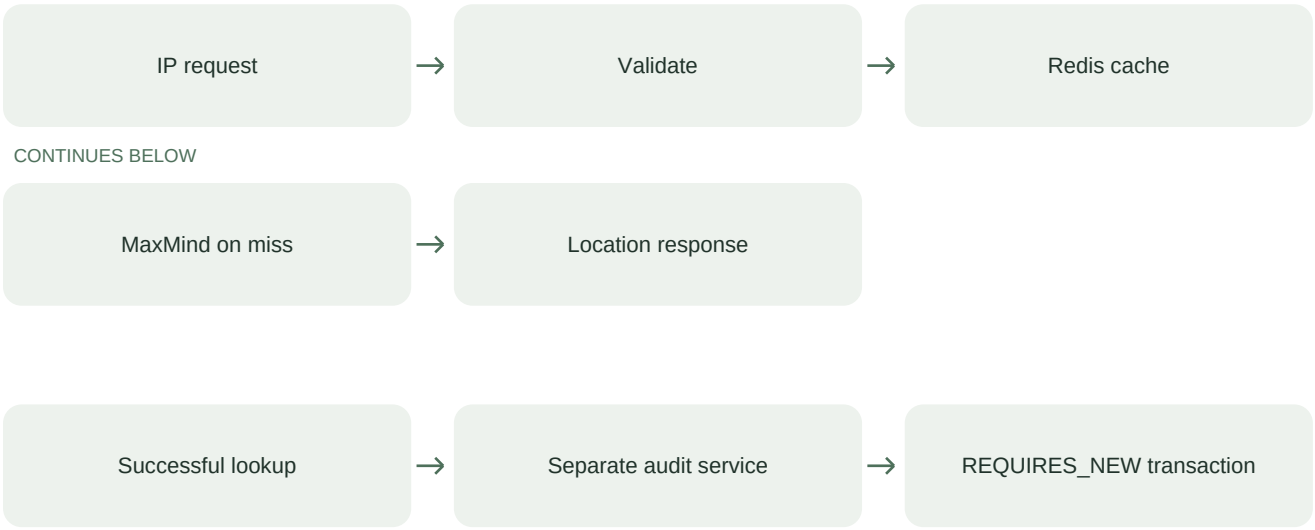
Create a held-out set: ten factual questions, five paraphrases, five unanswerable questions, and several cross-document questions. Label expected IDs/pages before tuning. Compare configurations on the same set and keep the suggested demo questions separate. A failure table should contain question, expected source, returned source, score, outcome, and proposed fix.

If evidence never reaches the model, changing the answer prompt will not fix retrieval. If retrieval succeeds but answers fail, investigate context, model behavior, and citation support. Ask a synonym-heavy query in

hash mode and compare with an exact phrase. Never interpret similarity 0.8 as 80 percent answer accuracy.

10. Location service: the accurate story

The service receives an IP and returns its local MaxMind record, with Redis cache-aside and SQL usage logging. It does not obtain GPS, locate a phone precisely, or search nearby businesses. No replacement database was introduced in this upgrade.



Component	Responsibility
LocationController	Receive requests and return HTTP responses
GeoIPServiceImpl	Cache-aside and local database lookup
MaxMindGeoIpConfig	Construct the database reader
LocationResponse	City, state, country, coordinates, message
ApiUsageLoggingService	Persist audit data separately
GlobalExceptionHandler	Map application errors to responses

A cache hit returns the saved response. A miss queries MaxMind and attempts to cache it for 24 hours. Redis read/write failures do not prevent a successful local lookup. The cache is an optimization, not the authoritative source. Database updates can leave cached records stale until expiration; versioned prefixes or invalidation address this.

The implementation uses InetAddress.getByName, which can resolve hostnames as well as numeric addresses. Strict literal-IP parsing would be a future improvement. Do not claim the current code rejects every hostname or special address without adding and testing that rule.

Coordinates are approximate network metadata. VPNs, carrier gateways, and corporate networks can make them differ from physical location. Private IPs are not globally unique. Missing records should produce a clear not-found response rather than invented coordinates.

REQUIRES_NEW suspends an existing transaction and starts a separate one for logging. Calls must pass through a separate Spring-managed bean so proxy interception applies. Calling a transactional method on the same object can bypass that interception. Tests verify audit commits despite caller rollback and logging failure does not replace a successful response.

Earlier local measurements showed direct MaxMind faster than a Redis round trip. Redis adds serialization and network overhead. Cache-aside demonstrates caching and resilience, not an automatic speed improvement. Do not repeat the unsupported 90 percent speedup claim.

Exercise: draw cache hit, miss, Redis outage, and unknown-IP paths. Identify which dependency is essential. Explain why a large IP database still cannot provide exact GPS. Inspect its actual file size for deployment planning and check current provider terms before redistribution.

11. Tests: prove behavior

A unit test isolates a rule with controlled collaborators. An integration test uses real dependencies. An end-to-end test drives HTTP or UI. Mocked Redis does not prove Lua atomicity. A load test without assertions may generate traffic without checking correctness.

Project	Check	Why it matters
Sentinel	100 concurrent requests admit exactly 10	Detect races and duplicate-member loss
Sentinel	Burst expiry followed by sustained rejection	Prove two-window interaction
Sentinel	Timeout, restart, expiration, blocklist retry	Exercise failure and recovery
Folio	Cross-user list/query/delete denial	Verify authorization beyond login
Folio	Corrupt-index rebuild and dimension rejection	Preserve index integrity
Folio	Samples, abstention, scope, protected jobs/files	Cover new workspace contract
Location	Audit isolation and cache/log failures	Preserve valid lookup responses

Run mvn test for Sentinel with Docker running; Testcontainers starts real Redis. The opt-in benchmark is separate and normally skipped. For Folio, install requirements.txt in a virtual environment and run python -m unittest discover -s tests -v. A skipped test is not a passed test.

Benchmark records should include commit, machine, dependencies, concurrency, duration, warmup, request mix, and raw results. Separate allowed, rejected, and failed calls. Report percentiles with

throughput. Browser previews are functional checks, not controlled benchmarks.

Exercise: in a disposable branch, change a quota comparison from greater-than-or-equal to greater-than. Predict which test fails, run it, and restore the code. This makes the assertion's purpose concrete.

12. Run and deploy

Folio: create a virtual environment, install requirements.txt, and start uvicorn app.main:app --host 127.0.0.1 --port 8000 from the repository root. Hash/mock modes need no paid key. The sample needs no login. Private uploads need Google OAuth with matching callback/frontend URLs. Use the environment example, but never commit the real .env file.

Sentinel: provide MySQL, Redis, datasource settings, and Redis settings. Configure RATE_LIMITER_ADMIN_KEY with a fresh random secret of at least 32 characters. Run mvn spring-boot:run, or mvn package followed by java -jar on the built jar. Spring Boot serves its frontend without a separate build. The benchmark Compose stack uses disposable MySQL storage and is not a persistent production deployment.

Concern	Folio	Sentinel	Location
Persistent data	SQLite, PDFs, vectors, metadata together	MySQL policies/logs; temporary Redis quotas	MaxMind file and SQL audit data
Secrets	JWT, OAuth state, Google/provider credentials	Admin, SQL, Redis credentials	Configured SQL/Redis credentials
Public boundary	HTTPS and owner scope	HTTPS and protected administration	Validation and approximate-location explanation
Scaling limit	Single-worker file-backed index	SQL authentication/logging and Redis capacity	Local reader and cache topology

Before deployment verify persistent disks, strong secrets, HTTPS, origins, callbacks, connectivity, and backup restoration. Smoke-test the deployed version. Local tests do not verify cloud OAuth, provider billing, TLS, or hosting settings. A source push and a verified deployment are different events.

Add infrastructure only for a measured need. Folio's next scaling step is durable jobs and safe shared storage. For Sentinel, profile SQL authentication/logging before assuming Redis is the bottleneck. For location, measure caching under the real network topology.

13. Frontend engineering

HTML gives semantic structure, CSS supplies layout, and JavaScript manages interactions. Both apps serve assets from their backend. The important work is asynchronous requests, state, errors, accessibility, and truthful data display, not merely adding a framework name.

Folio uses a paper/sage design for reading; Sentinel uses a dark dashboard with distinct outcome colors. Responsive layouts, labelled controls, native keyboard-accessible elements, empty states, and reduced-motion rules support usability. Decorative art uses CSS instead of large image dependencies.

fetch returns a response even for HTTP 400 or 500, so inspect response.ok. Network failures throw separately. Disable buttons during request batches to avoid duplicate actions. Stop upload polling at terminal states. Revoke temporary PDF object URLs to release memory.

Promise.all starts the dashboard batch concurrently from JavaScript's perspective. Browser connection limits and scheduling still affect arrival times; this is not the coordinated concurrency test. Browser p95 includes frontend, network, and backend time, unlike a service microbenchmark.

Exercise: test offline, expired login, empty library, and throttled states. Check that each explains the problem and permits recovery. Inspect narrow screens and navigate with Tab/Enter.

14. Interview walkthroughs

Sentinel: I built a Spring Boot limiter with Redis Lua for atomic concurrent decisions. Registered clients use hashed keys and two sliding windows, one for bursts and one for sustained usage. Both must pass before either receives a permit. The dashboard sends real traffic and exposes decisions. Real-Redis tests verify exact quotas. Redis outages fail open for availability, while authentication failures fail closed.

Folio: I built an authenticated PDF research workspace. It extracts pages, chunks text, embeds it, retrieves only owned documents, and supplies labelled context to a configurable answer provider. The UI shows excerpts and original pages. Uploads expose processing stages, and unsupported questions can abstain before a model call. A fictional public sample works without credentials. The file-backed single-worker pipeline is a known limitation.

Location: I built a public-IP lookup API backed by local MaxMind data. It returns approximate metadata with Redis cache-aside and fallback during cache failure. Usage logging uses a separate Spring bean and transaction. I measured the cache rather than assuming it was faster; direct lookup won in the local setup.

Question	Strong answer includes
Why Lua instead of separate commands?	Decision and writes must be atomic; commands can interleave
Why not trust forwarded headers?	Callers can spoof them; trusted proxies require explicit configuration
Why two windows?	Short spikes and sustained traffic are different behaviors

Question	Strong answer includes
Does RAG eliminate hallucination?	Retrieval, generation, and citation support can each fail
How is owner isolation enforced?	Server-derived IDs at list, query, file, job, and delete boundaries
Why overlap chunks?	Preserve boundary context at a duplication cost
Can Folio run four workers today?	Not as a supported safe configuration with current local indexing/jobs
Is IP latitude exact?	No; it describes network metadata, not live GPS
What does your benchmark measure?	Name workload, environment, path, outcomes, and excluded layers

Choose a concrete weakness when discussing future work. Examples: held-out retrieval evaluation, ingestion resource limits, trusted proxies, or measured SQL overhead. Do not claim planned features are implemented.

15. Learning and contribution plan

Sessions	Focus	Deliverable
1-2	HTTP, JSON, controllers, dependencies	Trace one request in each app
3-4	Sorted sets and Lua	Hand-simulate and reproduce the quota table
5-6	Keys and failure policy	Demonstrate create, disable, rotate, and outage
7-8	PDF chunks and embeddings	Draw ingestion and query pipelines unaided
9-10	Ownership and evaluation	Explain isolation tests; label held-out questions
11-12	Caching and transactions	Explain location failures and REQUIRES_NEW
13-14	UI state and responsive design	Fix one bug and record a demo
15-16	Tests and interview rehearsal	Run suites and explain each project

Use a cycle: read, predict, experiment, record, explain. Move faster when you can explain the mechanism, not merely when a command succeeds. Keep a failure journal: symptom, hypothesis, evidence, fix, and regression test.

Make your first independent changes small: add a negative ownership case, improve an error message, label a retrieval example, or improve an accessibility label. Then build a larger feature with requirements and tests. Explaining your own change deeply is better than memorising generated code.

16. Resume and demo checklist

Use a one-column resume with selectable text, familiar headings, and evidence-backed bullets. Give the upgraded projects prominence while describing the internship honestly. Do not promise universal ATS compatibility, arbitrary shortlist odds, production scale, or unsupported accuracy.

Prepare a three-minute demo: show a Sentinel rejection, inspect Folio evidence, and explain a location failure path. Show one meaningful test and one limitation per project. Without configured OAuth/provider credentials, use the labelled sample and state what it demonstrates.

Verify every number against its source. Explain your contribution, assistance used, relevant code, a failure you fixed, and the tradeoff accepted. Strong projects improve the application, but eligibility, timing, role requirements, competition, and hiring decisions still affect shortlisting.